



Express.js

פריימוורק גמיש לפיתוח אפליקציות Fullstack



מה זה Express.js?

Express.js הוא פריימוורק מינימליסטי ל-Node.js.

משמש ליצירת:

- APIs
- אפליקציות Fullstack
- מערכות Microservices

מאפיינים מרכזיים:

- קל משקל, גמיש, ומודולרי.
- תומך במגוון רחב של Middleware.

התקנת החבילה:

```
npm install express
```

ייבוא החבילה בקובץ server.js:

```
import express from 'express'
```



יצירת שרת בסיסי

```
import express from 'express';

const PORT = 3000;

const server = express();

server.use(express.json()); // תמיכה בקריאה ושליחת json

server.listen(PORT, () => {

  console.log(Server running on http://localhost:${PORT});

});
```



מה הם Express.js Routes?

:METHOD

אופן הגדרת הבקשה: GET, POST, PUT, DELETE

Express.js Routes הם דרכים להגדיר איך האפליקציה מגיבה לבקשות HTTP ספציפיות.

:PATH

כתובת ה-URL (מסלול).

מבנה כללי:

```
server.METHOD(PATH, HANDLER);
```

:HANDLER

פונקציה שמטפלת בבקשה.

מהן פעולות HTTP?

פעולות (HTTP Methods) הן דרכים שבהן לקוח (Client) ושרת (Server) מתקשרים.

כל פעולה מייצגת כוונה מסוימת, כגון קבלת נתונים, יצירת נתונים, עדכון או מחיקה.

שימוש נפוץ	תיאור	פעולה
הצגת רשומות או דף מידע.	בקשה לקרוא או לקבל נתונים מהשרת.	GET
יצירת משתמש חדש, שליחת טפסים.	שליחת נתונים לשרת ליצירת רשומה חדשה.	POST
עדכון פרטי משתמש או נתונים קיימים.	עדכון רשומה קיימת בשרת (כולל החלפה מלאה של הנתונים).	PUT
מחיקת משתמש, רשומה או פריט.	מחיקת נתונים מהשרת.	DELETE



דוגמה ל Route בסיסי

הסבר:

מסלול זה מאזין לבקשות GET לכתובת ./.
שולח תגובה טקסטואלית לדפדפן.

```
import express from 'express';

const PORT = 3000;

const server = express();

server.use(express.json()); // תמיכה בקריאה ושליחת json

server.get('/', (req, res) => {

  res.send('Welcome to the Home Page!');

});

server.listen(PORT, () => {

  console.log(Server running on http://localhost:${PORT});

});
```



Route עם פרמטרים דינמיים

הסבר:

id: הוא פרמטר דינמי וניתן לגשת אליו דרך req.params.id.

```
import express from 'express';
const PORT = 3000;
const server = express();
server.use(express.json()); //תמיכה בקריאה ושליחת json
server.get('/products/:id', (req, res) => {
  const { id } = req.params;
  res.send(`Product ID: ${id}`);
});
server.listen(PORT, () => {
  console.log(Server running on http://localhost:${PORT});
});
```

Route עם שאילתות (Query Parameters)

```
import express from 'express';
```

```
const PORT = 3000;
```

```
const server = express();
```

```
server.use(express.json()); //תמיכה בקריאה ושליחת json
```

```
server.get('/search', (req, res) => {
```

```
  const { term, category } = req.query;
```

```
  res.send(`Search term: ${term}, Category: ${category}`);
```

```
});
```

```
server.listen(PORT, () => {
```

```
  console.log(Server running on http://localhost:${PORT});
```

```
});
```

הסבר:

שאילתות מועברות לאחר ? ב-URL:

/search?term=nodejs&category=programming



מסלולים מרובי שיטות HTTP

הסבר:

server.route - מקבצת פעולות שונות באותו מסלול.

```
server.route('/users')  
  .get((req, res) => res.send('Get all users'))  
  .post((req, res) => res.send('Create a new user'))  
  .put((req, res) => res.send('Update all users'))  
  .delete((req, res) => res.send('Delete all users'));
```

Microservices



מה הם Microservices?

גישה לפיתוח תוכנה המחלקת את המערכת לשירותים קטנים ונפרדים לכל שירות:

- בעל אחריות ספציפית.
- עצמאי, ניתן לפריסה בנפרד.
- מתקשר עם שירותים אחרים דרך APIs

למה Express.js מתאים למיקרו-שירותים?

1. קלילות: Express הוא מינימליסטי ואינו דורש משאבים רבים.
2. API-first: תומך ביצירת APIs פשוטים וגמישים.
3. מודולריות: מאפשר לכל מיקרו-שירות להיות יחידה עצמאית.
4. תמיכה בקהילת הקוד הפתוח: המון תוספים ותמיכה חזקה.



יצירת microservice באמצעות Express Router

//file: products/router.js

```
import express from 'express';  
const productRouter = express.Router();
```

productRouter

```
  .get('/', (req, res) => { res.send('List of all products'); })  
  .get('/:id', (req, res) => { res.send(`Product ID: ${req.params.id}`); })  
  .post('/', (req, res) => { res.send('Product created'); })
```

```
export default productRouter;
```

//file: server.js

```
import productRouter from './routes/products.js';  
server.use('/products', productRouter);
```

Middlewares



מה זה Middleware?

Middleware הוא פונקציה שמבוצעת במהלך מחזור החיים של בקשה-תגובה ב-Express.js המיועדת עבור:

- ניתוח נתונים בבקשה.
- הוספת לוגיקה לפני שליחת תגובה.
- טיפול בשגיאות.



איך Middleware עובד?

```
(req, res, next) => { ... }
```

כל Middleware מקבל שלושה פרמטרים:

- req: האובייקט של הבקשה (Request).
- res: האובייקט של התגובה (Response).
- next: פונקציה להעברת הבקשה ל-Middleware הבא.



סוגי Middleware

1. Built-in Middleware (מובנה) ונכלל ב-Express.
2. Third-party Middleware הקיימות בחבילות כגון cors ו-morgan
3. Custom Middleware - פונקציות שאנחנו נכתוב



Built-in Middleware

```
server.use(express.json()); // JSON ניתוח
```

```
server.use(express.urlencoded({ extended: true })); // נתוני טפסים
```

```
server.use(express.static('public')); // קבצים סטטיים
```



Custom Middleware

הסבר:

Middleware זה מדפיס את שיטת הבקשה
וכתובת ה-URL.

```
function logger (req, res, next) {  
  
  console.log(`${req.method} ${req.url}`);  
  
  next(); // הבא ה-Middleware-המשך ל  
  
};  
  
server.use(logger);
```



שימוש ב-Middleware ב-Route

הסבר:

```
server.get('/user', logger, (req, res) => {  
  
    res.send('User route');  
  
});
```

Middleware זה מדפיס את שיטת הבקשה
וכתובת ה-URL.



Third-party Middleware

התקנת ושימוש ב-Middleware חיצוני

```
npm install morgan
```

דוגמה לשימוש

```
import morgan from 'morgan';
```

```
server.use(morgan('tiny')); // לוגים לבקשות נכנסות
```

הסבר

חבילה זו מאפשרת לבצע הדפסת לוגים ושמירתם בקובץ `txt`.



Middleware לטיפול בשגיאות

Middleware המיועד לטיפול בשגיאות מקבל 4 פרמטרים:

```
function errorHandler (err, req, res, next) {  
  
    console.error(err.message);  
  
    res.status(500).send('Something broke!');  
  
};  
  
server.use(errorHandler);
```



איך Middleware עובד במחזור בקשה-תגובה?

1. בקשה נכנסת לשרת.
2. Middleware הראשון מופעל.
3. אם ה-Middleware לא שלח תגובה, הבקשה מועברת ל-Middleware הבא.
4. התהליך נמשך עד שמתקבלת תגובה.



טיפים לעבודה עם Middleware

1. סדר ה-Middleware חשוב – הם מבוצעים בסדר שבו הם מוגדרים.
2. תמיד השתמש ב-`next()` להעברת הבקשה הלאה.
3. Middleware לטיפול בשגיאות צריך להופיע בסוף.
4. השתמש ב-Middleware ייעודי לכל מסלול כשנדרש.



דוגמה מלאה

```
import express from 'express';

import morgan from 'morgan';

const server = express();

// Middleware מובנים
server.use(express.json());

server.use(express.static('public'));

// Middleware חיצוני
server.use(morgan('tiny'));

// Middleware מותאם
server.use((req, res, next) => {

    console.log('Custom Middleware: Request received');

    next();

});
```

```
// ייעודי Middleware מסלול עם

server.get('/user', (req, res, next) => {

    console.log('Middleware for /user route');

    res.send('User route');

});

// טיפול בשגיאות

server.use((err, req, res, next) => {

    console.error(err.stack);

    res.status(500).send('Error occurred!');

});

// הפעלת השרת

server.listen(3000, () => {

    console.log('Server running on http://localhost:3000');

});
```